

# Chall AES CTR

## Analisis

sc =

```
from Crypto.Cipher import AES
from Crypto.Util import Counter
import os

FLAG = b"CTF{r3used_n0nc3_XOR_br34ks_str34m_clph3r}" #anggap gak ada
:v

HINT = (
    b"Halo peserta CTF! Pesan ini hanyalah plaintext contoh yang "
    b"sengaja dienkripsi memakai nonce yang sama dengan flag. "
    b"Selamat mencoba mencari kelemahannya, semoga berhasil!"
)

def encrypt_ctr(key: bytes, nonce: bytes, plaintext: bytes) -> bytes:

    ctr = Counter.new(64, prefix=nonce, initial_value=0)
    cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
    return cipher.encrypt(plaintext)

def main():

    key = os.urandom(16) # AES-128
    nonce = os.urandom(8) # nonce 8 byte

    ct_flag = encrypt_ctr(key, nonce, FLAG)
    ct_hint = encrypt_ctr(key, nonce, HINT)

    with open("output_aes-ctr_chall_claude.txt", "w") as f:
        f.write(f"hint_plaintext = {HINT!r}\n")
        f.write(f"ct_flag = {ct_flag.hex()}\n")
        f.write(f"ct_hint = {ct_hint.hex()}\n")

    print("[+] Selesai! Cek output.txt")
```

```
if __name__ == "__main__":  
    main()
```

output =

hint\_plaintext = b'Halo peserta CTF! Pesan ini hanyalah plaintext contoh yang sengaja dienkripsi memakai nonce yang sama dengan flag. Selamat mencoba mencari kelemahannya, semoga berhasil!'

ct\_flag =

254626eccceadcb2b0a41fba2ef02cf0e7afae1d50da82606b5d4bcf979e  
a47882727b5357640ad0af4d

ct\_hint =

2e730cf89ea9ccb2b0b234b53edd1b8599d7b12a7cd99e73365851b08  
c8bb832d773455846640e82b45e0841e668b4009513887b6bd76aa0d4  
11e91358efc0afba18bc8f3b0ae6c7315e316e0944efbebbb9bb179ebce  
79b15f872b45f6c020bfc5fe4803fbac1d98a3efa853c066aaa87ce3d92  
832da24838a839e9aaf86771d8a1ad5118928bb1e195de244f2c0db770  
93c5b96df8429e6de06e8c1454e66b96fb43f2c7e6133303

chall ini saya dapat dari claude yang di,mana saya minta claude untuk membuatkan chall AES mode CTR dengan kerentanan dimana nonce digunakan berulang.

di sc chall kita bisa tau bahwa flag dan hint di enkripsi menggunakan nonce dan key yang sama,dimana seharusnya kita harus membedakan nonce jika mau mengenkripsi plain text yang berbeda.

```
key = os.urandom(16)    # AES-128  
nonce = os.urandom(8)  # nonce 8 byte  
  
ct_flag = encrypt_ctr(key, nonce, FLAG)  
ct_hint = encrypt_ctr(key, nonce, HINT)
```

## Solusi

solver =

```
ct_flag =
bytes.fromhex("254626eccceadcb2b0a41fba2ef02cf0e7afae1d50da82606b5d4bcf
979ea47882727b5357640ad0af4d")
ct_hint =
bytes.fromhex("2e730cf89ea9ccb2b0b234b53edd1b8599d7b12a7cd99e73365851b0
8c8bb832d773455846640e82b45e")
plain_hint_42bytes =
bytes.fromhex('48616c6f2070657365727461204354462120506573616e20696e6920
68616e79616c616820706c61696e')

def xor(a,b):
    return bytes([x^y for x,y in zip(a,b)])

keystream = xor(ct_hint,plain_hint_42bytes)
print(keystream)
print(len(keystream))

flag = xor(keystream,ct_flag)
print(flag)
```

jadi yang kita ketahui adalah:

$ct\_flag = plain\_flag \oplus K1$

$ct\_hint = plain\_hint \oplus K1$

karena keystream yang di gunakan untuk mengenkripsi hint dan flag sama dan kita di kasih tau plaintext dari hint (yang ada di file output)maka kita bisa mendapatkan keystream dengan cara

$$\begin{aligned} keystream &= ct\_hint \oplus plain\_hint \\ &= (plain\_hint \oplus keystream) \oplus plaint\_hint \\ &= keystream \quad (\text{karena sifat xor dimana } (a \oplus b) \oplus a = b) \end{aligned}$$

disini untuk bisa mendekripsikan flag maka kita harus mendapat keystream yang sama dengan proses enkripsi, untuk dapat keystream yang sama maka kita bisa mengxorkan 42 bytes dari plain\_hint dengan 42 bytes dari ct\_hint supaya mendapat keystream yang ukuran

dan nilai nya sesuai dengan flag.mengapa kita harus mengxorkan 42 bytes dari ct\_hint dengan 42 bytes plain\_hint?karena panjang flag kita adalah 42 bytes . Di AES mode CTR nilai dari keystream kita terus berubah di tiap blok.oleh karena itu kita juga harus mendapat kan nilai dari keystream yang mengenkripsi 42(2 blok + 10 bytes) bytes flag untuk mendeskripsikan lagi flag kita.karena nonce dan key yang di gunakan untuk mengenkripsi plain\_hint dan plain\_flag sama,maka kita bisa simpulkan bahwa keystream yang di gunakan untuk mengenkripsi 42 bytes plain\_hint == keystream yang digunakan untuk mengenkripsi 42 bytes flag

```
ct_flag[0:42] = plain_flag[0:42] ^ keystream(x)
```

```
ct_hint[0:42] = plain_hint[0:42] ^ keystream(x) #keystream nya sama
```

jadi setelah tau keystream yang mengenkripsi 42 bytes plain\_hint kita bisa mengetahui plain\_flag

```
plain_flag = ct_flag ^ keystream
```

```
            = (plain_flag ^ keystream) ^ keystream
```

```
            = plain_flag
```

```
flag = b'CTF{r3used_n0nc3_XOR_br34ks_str34m_c1ph3r}'
```

